

Preparación de datos Ivy GAP para su deconvolución

Es necesario pasar los datos a CPM

1. Cargar paquetes necesarios

```
library(tidyverse)
library(SummarizedExperiment)
library(rstatix)
library(ggpubr)
library(dplyr)
library(biomaRt)
```

2. Cargar los archivos

```
#Cargar IVYGAP en TPM (este es el ARCHIVO FILTRADO)
ivygap <- readRDS ("IVYGAP.RDA")
datos_ivy_TPM <- assay(ivygap) # Matriz de TPM de expresión

#Cargar archivo csv de ivygap -> ARCHIVO CRUDO
csv_ivy <- read.csv("raw_fpkm.csv")
```

3. Filtrar el csv Hay muestras que no nos interesan. Deben quedar las muestras que coincidan entre el archivo de los TPM ya filtrados y el archivo csv crudo

```
#Encontrar las coincidencias entre las columnas de los dos dataframes
#La columna que tienen en común es el rna well
csv_filtrado <- match(colnames(datos_ivy_TPM), csv_ivy$rna_well)
csv_ivy = csv_ivy[csv_filtrado,]
#filtra csv_ivy de acuerdo a los indices de coincidencia de csv_filtrado,
#reasignando el data frame

colnames(csv_ivy) #Propiedades de las muestras
colnames(datos_ivy_TPM) #Nombres de las muestras

#comprobaciones
mean(colnames(datos_ivy_TPM) %in% csv_ivy$rna_well) #1
mean(csv_ivy$rna_well %in% colnames(datos_ivy_TPM)) #1
mean(colnames(datos_ivy_TPM) == csv_ivy$rna_well) #1
```

4. Obtener la library size Para calcular LS: $LS = (rnaseq_total_reads * rnaseq_percent_reads_aligned_to_mrna) / 100$

```

csv_ivy$Library_size <- (csv_ivy$rnaseq_total_reads *
                        csv_ivy$rnaseq_percent_reads_aligned_to_mrna) / 100

library_size <- (csv_ivy$rnaseq_total_reads *
                csv_ivy$rnaseq_percent_reads_aligned_to_mrna) / 100

```

5. Bucle for

```

#Comprobar que tienen dimensiones compatibles
library_size = as.numeric(library_size)

if (length(library_size) != ncol(datos_ivy_TPM)) {
  stop("Error: 'library_size' no tiene la misma cantidad de muestras que 'datos_ivy_TPM'.")
}

#crear una matriz llamada ivy_counts con las mismas dimensiones que datos_ivy_TPM
#y con los mismos nombres de filas y columnas, pero llena de NAs
ivy_counts = matrix(NA, nrow = nrow(datos_ivy_TPM), ncol = ncol(datos_ivy_TPM),
                    dimnames = list(rownames(datos_ivy_TPM),
                                    colnames(datos_ivy_TPM)))

# Convertir TPM a counts
for (i in 1:ncol(ivy_counts)) {
  ivy_counts[, i] = round((datos_ivy_TPM[, i] * library_size[i]) / 10^6)
}

ivy_counts_df <- as.data.frame(ivy_counts)

```

6. Añadir el nombre de la columna “gene”

```

ivy_counts_df %>% rownames_to_column("Gene") -> ivy_counts_df
ivy_counts_df[1:5, 1:5]

```

7. Obtener la longitud promedio de los transcritos

```

mart <- useMart("ensembl", dataset = "hsapiens_gene_ensembl")

transcript_lengths <- getBM(
  attributes = c("external_gene_name", "transcript_length"),
  mart = mart)

#Cambiar nombre de los nombres de genes a "Gene" (como en ivygap)
colnames(transcript_lengths)[1] <- "Gene"

#Obtener la longitud promedio
sum(duplicated(transcript_lengths$Gene)) #238183

avg_lengths <- aggregate(transcript_length ~ Gene, data = transcript_lengths,
                        FUN = mean)

sum(duplicated(avg_lengths$Gene)) #0

```

10. Reordenar los genes como en Ivygap

```
gene_order <- match(ivy_counts_df$Gene, avg_lengths$Gene)
#encuentra el índice de cada gen dentro de avg_length

avg_length_ordenado <- avg_lengths[gene_order, ]

any(is.na(avg_length_ordenado)) #FALSE
mean(is.na(avg_length_ordenado)) #0.001078232 -> genes ivygap que no están en la BD
avg_length_ordenado <- na.omit(avg_length_ordenado)

mean(avg_length_ordenado$Gene%in%ivy_counts_df$Gene) #1
mean(ivy_counts_df$Gene%in% avg_length_ordenado$Gene) #0.99
ivy_counts_filtered <- ivy_counts_df[ivy_counts_df$Gene %in% avg_length_ordenado$Gene, ]
#filtra las filas de ivy_counts_df y se queda solo con aquellas cuyo gen
 #(Gene) está también presente en gene_coordinates_ordenado

#Para calcular el porcentaje de coincidencias exactas
mean(ivy_counts_filtered$Gene == avg_length_ordenado$Gene) #1
```

11. Multiplicar los counts por la longitud de los transcritos

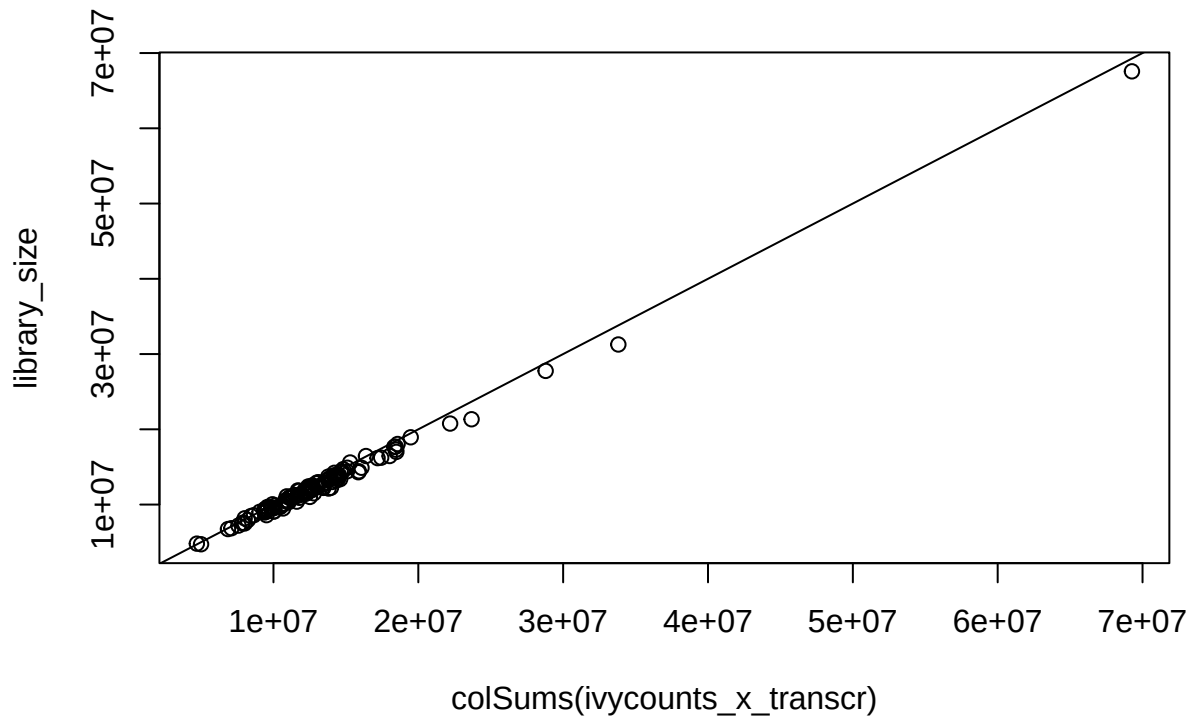
```
ivy_counts_filtered[, 1]
#Son los genes. Como no son numéricos, no puede realizarse la operación.

counts_numeric <- ivy_counts_filtered[, -1]
avg_longitud <- avg_length_ordenado$transcript_length

ivycounts_x_transcr <- round(counts_numeric* (avg_longitud/1000))

rm(counts_numeric)

#Gráfico para comprobar:
plot(colSums(ivycounts_x_transcr), library_size)
abline(0,1)
```



```
raw_counts <- cbind(Gene = ivy_counts_filtered$Gene, ivycounts_x_transcr)
```

Normalizar a CPM

```
#Pasar a CPM. Función

library(edgeR)
cpm_matrix <- cpm(ivycounts_x_transcr)
CPM <- cbind(Gene = ivy_counts_filtered$Gene, cpm_matrix)
CPM <- as.data.frame(CPM)
```

13. Requisitos de formato

```
#Tab-delimited tabular input format (.txt) with no double
#quotations and no missing entries.
any(is.na(CPM)) #FALSE
any(CPM == "") #FALSE

#Genes in column 1; Mixture labels (sample names) in row 1
CPM[1:5, 1:5] #Se cumple
```

#Given the significant difference between counts (e.g., CPM) and gene length-normalized expression data (e.g., TPM) we recommend that the signature matrix and mixture files be represented in the same normalization space whenever possible.

#Data should be in non-log space. Note: if maximum expression value is less than 50; CIBERSORTx will assume that data are in log space, and will anti-log all expression values by 2x.

```
sapply(CPM, class) #Gene es character
max_value <- max(sapply(CPM, function(col) if (is.numeric(col)) max(col, na.rm = TRUE) else NA), na.rm = TRUE)
print(max_value) #95968.27

#CIBERSORTx will add an unique identifier to each redundant gene symbol, however
#we recommend that users remove redundancy prior to file upload.
sum(duplicated(colnames(CPM))) #0
```

CIBERSORTx performs a feature selection and therefore typically does not use all genes in the signature matrix. It is generally ok if some genes are missing from the user's mixture file. If less than 50% of signature matrix genes overlap, CIBERSORTx will issue a warning.

```
cout <- read.table("Cout_counts_remix.txt", header = TRUE, sep = "\t")

mean(cout$GeneSymbol %in% CPM$Gene) #99%.
#No es un 100% por los genes de ivygap que quitamos al no estar presentes en
#la base de datos con los transcritos
mean(CPM$Gene %in% cout$GeneSymbol) #77.3%
```

14. Guardar

```
write_tsv(CPM, "Ivy_CPM.txt")
```